MINI PROJECT: Customer Orders ETL Pipeline (Spark + Scala)

Goal

Build an ETL pipeline that:

1. Reads customers + orders CSVs
2. Cleans data
3. Joins
4. Computes customer KPIs
5. Supports incremental loads
6. Writes final output as Parquet

---

1. Folder Structure

```
SparkETLProject/
│
├── build.sbt
├── README.md
│
├── src/
│   ├── main/
│   │   ├── resources/
│   │   │     └── application.conf
│   │   │
│   │   └── scala/
│   │         ├── Main.scala
│   │         ├── config/ConfigLoader.scala
│   │         ├── utils/SparkUtils.scala
│   │         ├── utils/TransformUtils.scala
│   │         └── etl/ETLPipeline.scala
│   │
│   └── test/
```

```
|          └── sample tests…
|
└── data/
       ├── customers.csv
       └── orders.csv
```

---

2. build.sbt

```
name := "SparkETLProject"
version := "1.0"
scalaVersion := "2.12.17"

libraryDependencies ++= Seq(
  "org.apache.spark" %% "spark-core" % "3.4.1",
  "org.apache.spark" %% "spark-sql" % "3.4.1",
  "com.typesafe" % "config" % "1.4.2"
)
```

---

3. application.conf (Configuration File)

```
app {
  input.customers = "data/customers.csv"
  input.orders = "data/orders.csv"

  output.dir = "output/final"
  incremental.column = "order_date"
}
```

---

4. Sample Data

customers.csv

```
customer_id,first_name,last_name,age,city
1,John,Doe,30,New York
2,Mary,Smith,22,Texas
3,Peter,Jones,45,Florida
```

orders.csv

```
order_id,customer_id,amount,order_date
101,1,250,2024-01-10
102,1,150,2024-02-15
103,2,300,2024-01-18
104,3,500,2024-01-20
```

---

5. ConfigLoader.scala

```
package config

import com.typesafe.config.ConfigFactory

object ConfigLoader {
  private val config = ConfigFactory.load()

  val customersPath: String = config.getString("app.input.customers")
  val ordersPath: String = config.getString("app.input.orders")
  val outputDir: String = config.getString("app.output.dir")
  val incrementalCol: String =
config.getString("app.incremental.column")
}
```

---

6. SparkUtils.scala

```
package utils
```

```
import org.apache.spark.sql.SparkSession

object SparkUtils {

  def getSpark(appName: String): SparkSession = {
    SparkSession.builder()
      .appName(appName)
      .master("local[*]")  // local mode
      .getOrCreate()
  }

}
```

---

7. TransformUtils.scala

```
package utils

import org.apache.spark.sql.DataFrame
import org.apache.spark.sql.functions._

object TransformUtils {

  def cleanCustomers(df: DataFrame): DataFrame = {
    df.withColumn("first_name", initcap(trim(col("first_name"))))
      .withColumn("last_name", initcap(trim(col("last_name"))))
      .withColumn("full_name", concat_ws(" ", col("first_name"),
col("last_name")))
  }

  def cleanOrders(df: DataFrame): DataFrame = {
    df.withColumn("amount", col("amount").cast("double"))
      .withColumn("order_date", to_date(col("order_date")))
  }
```

```
  def computeCustomerKPIs(customers: DataFrame, orders: DataFrame):
DataFrame = {
    val aggOrders = orders.groupBy("customer_id")
      .agg(
        sum("amount").as("total_spent"),
        count("*").as("total_orders"),
        max("order_date").as("last_order_date")
      )

    customers.join(aggOrders, Seq("customer_id"), "left")
  }
}
```

---

8. ETLPipeline.scala

```
package etl

import config.ConfigLoader
import utils.{SparkUtils, TransformUtils}
import org.apache.spark.sql.functions._

object ETLPipeline {

  def run(): Unit = {

    val spark = SparkUtils.getSpark("CustomerOrdersETL")

    import spark.implicits._

    // ----------------------------
    // 1. Extract
    // ----------------------------
    val customersDF = spark.read
      .option("header", "true")
      .csv(ConfigLoader.customersPath)
```

```
    val ordersDF = spark.read
      .option("header", "true")
      .csv(ConfigLoader.ordersPath)

    // ------------------------------
    // 2. Transform
    // ------------------------------
    val cleanCustomers = TransformUtils.cleanCustomers(customersDF)
    val cleanOrders = TransformUtils.cleanOrders(ordersDF)

    val finalDF = TransformUtils.computeCustomerKPIs(cleanCustomers,
cleanOrders)

    // ------------------------------
    // 3. Load
    // ------------------------------
    finalDF.write
      .mode("overwrite")
      .parquet(ConfigLoader.outputDir)

    println("ETL Completed Successfully!")
  }
}
```

---

9. Main.scala

```
import etl.ETLPipeline

object Main {
  def main(args: Array[String]): Unit = {
    ETLPipeline.run()
  }
}
```

---

10. How to Run

In IntelliJ

1. Load sbt project
2. Create a run configuration for `Main`
3. Press Run

Spark Submit

```
spark-submit --class Main
target/scala-2.12/sparketlproject_2.12-1.0.jar
```
